Python E-Course

Module 11 — Segmentation & Thresholding

Detailed Notes

Module Overview

Level: Segmentation

Duration: ~1.5 weeks

Prerequisites: Modules 1-10

What You'll Learn

- Threshold images globally and adaptively.
- Use Otsu's method for automatic global thresholds.
- Run watershed for touching-object segmentation.
- Use GrabCut for interactive foreground extraction.

Lessons

#	Lesson	Duration
1	Binary Thresholding	25 min
2	Otsu's Method	25 min
3	Adaptive Thresholding	30 min
4	Watershed	35 min
5	GrabCut (Interactive Foreground)	30 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 12: Contours & Shape Analysis

Lesson 1: Binary Thresholding

Module: 11 — Segmentation & Thresholding

Duration: 25 minutes

Difficulty: Beginner

Learning Objectives

- Apply cv2.threshold with the five common type flags.
- Pick a sensible threshold from a histogram.
- Use binary masks as ROI selectors.

Prerequisites

- Modules 1-10

1. Why This Matters

Thresholding is the simplest segmentation method — turn a grayscale image into a binary "foreground vs background" mask. Despite its simplicity, it's surprisingly effective when the histogram is bimodal.

2. Concept

2.1 The function

```
ret, dst = cv2.threshold(src, thresh, maxval, type)
```

- src — grayscale uint8 image.
- thresh — the threshold value.
- maxval — the value used for pixels above the threshold (255 for binary).
- type — one of the flags below.

2.2 Threshold types

Type	Behaviour
THRESH_BINARY	pixel > thresh → maxval, else 0
THRESH_BINARY_INV	pixel > thresh → 0, else maxval
THRESH_TRUNC	pixel > thresh → thresh, else unchanged
THRESH_TOZERO	pixel > thresh → unchanged, else 0
THRESH_TOZERO_INV	pixel > thresh → 0, else unchanged

The first two are the "make a mask" forms; the others are clipping operations.

2.3 Reading a histogram for a threshold

Plot the histogram (M6 L1). If you see two clear peaks (bimodal), pick a value in the valley between them. If the peaks aren't clear, use Otsu (next lesson).

2.4 Apply on color requires a choice

cv2.threshold works on a single channel. For color images:

- Convert to grayscale, threshold.
- Or convert to HSV and threshold on a channel (e.g., V to find bright regions).
- Or use cv2.inRange for multi-channel thresholding (M4).

2.5 The simple recipe

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)
```

3. Code Examples

Example 1: Basic threshold

```
import cv2
from skimage.data import coins

img = coins()
_, mask = cv2.threshold(img, 80, 255, cv2.THRESH_BINARY)
cv2.imwrite("coins_thresh80.png", mask)
print("foreground %:", (mask > 0).mean() * 100)
```

Expected Output: coins_thresh80.png shows white coins on black background. foreground %: ~25 (varies).

Example 2: Try several thresholds

```
import cv2, numpy as np
from skimage.data import coins

img = coins()
for t in [60, 90, 120, 150]:
    _, mask = cv2.threshold(img, t, 255, cv2.THRESH_BINARY)
    cv2.imwrite(f"coins_t{t}.png", mask)
    print(f"t={t:3d}  fg={ (mask > 0).mean()*100:5.1f}%")
```

Expected Output: Lower thresholds → more white (over-segmenting); higher thresholds → less white (loses coin tops).

Example 3: All five threshold types

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
types = [cv2.THRESH_BINARY, cv2.THRESH_BINARY_INV, cv2.THRESH_TRUNC,
         cv2.THRESH_TOZERO, cv2.THRESH_TOZERO_INV]
names = ["BINARY", "BINARY_INV", "TRUNC", "TOZERO", "TOZERO_INV"]
for t, n in zip(types, names):
    _, out = cv2.threshold(img, 127, 255, t)
    cv2.imwrite(f"types_{n}.png", out)
```

Expected Output: Five files showing the different behaviors — BINARY makes a hard mask, TRUNC caps bright values, TOZERO blackens dark values, etc.

Example 4: Use the mask as ROI selector

```
import cv2
from skimage.data import coins

img = coins()
_, mask = cv2.threshold(img, 80, 255, cv2.THRESH_BINARY)
# show only the coins, blacken the background
result = cv2.bitwise_and(img, img, mask=mask)
cv2.imwrite("coins_masked.png", result)
```

Expected Output: Coin image where the background is solid black; the coins themselves are preserved at original intensity.

4. Parameter Sensitivity

Param	Effect
thresh	low → more "foreground"; high → less
type	binary vs clipping behaviour
maxval	what to set pixels above thresh to (usually 255)

5. Common Mistakes

Mistake	What Happens	Fix
Passing a color image	Error or wrong result	Convert to gray first
Threshold on raw uneven lighting	Black blobs on one side, all-white on the other	Use adaptive (L3)
Forgetting ret is the threshold value	TypeError when unpacking	Always ret, mask = cv2.threshold(...)

6. Practice Tasks

- Threshold coins() at 60, 90, 120. Compare.
- Apply THRESH_TOZERO at 100 to camera().
- Use a threshold mask to keep only the coins, blacken background.

7. Key Takeaways

- cv2.threshold(src, thresh, maxval, type) returns (ret, dst).
- Five threshold types — binary forms make masks; others clip values.
- Picking thresh from a bimodal histogram works; otherwise use Otsu.
- Always convert color to gray first.

8. What's Next

Otsu's method — automatic threshold picking.

Lesson 2: Otsu's Method

Module: 11 — Segmentation & Thresholding

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Run Otsu thresholding with cv2.THRESH_OTSU.
- Explain the principle (maximise between-class variance).
- Recognise when Otsu fails (non-bimodal histograms).

Prerequisites

- Module 11 Lesson 1

1. Why This Matters

Otsu picks the optimal global threshold automatically — no human guessing. It's the right default when the histogram is bimodal (clear foreground / background separation).

2. Concept

2.1 The intuition

Imagine sliding a candidate threshold from 0 to 255. At each value, split the histogram into two groups: pixels below, pixels above. Compute the variance between the two groups (or equivalently, minimise within-group variance). The threshold that maximises between-class variance is Otsu's choice.

2.2 OpenCV API

```
otsu_value, mask = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

Pass 0 as the threshold — Otsu's algorithm picks it. The function also returns the chosen value in otsu_value.

You can combine with other flags:

```
cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
```

2.3 Pre-blur for stability

Like every threshold method, Otsu is noise-sensitive. A small Gaussian blur first stabilises the threshold:

```
blur = cv2.GaussianBlur(gray, (5, 5), 0)
otsu, mask = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

2.4 When Otsu fails

- Unimodal histogram (uniform image): no "valley" to pick → produces a meaningless mask.

- Uneven lighting: one global threshold can't fit; use adaptive (next lesson).
- More than 2 classes: Otsu only picks one boundary; use multi-level Otsu (not built into cv2; use skimage.filters.threshold_multiotsu).

2.5 Bimodality check

A quick test: compute histogram, find peaks, ratio of valley to peak. If the valley is at least 30% lower than both peaks, Otsu should work well.

3. Code Examples

Example 1: Basic Otsu

```
import cv2
from skimage.data import coins

img = coins()
otsu, mask = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print("Otsu chose:", otsu)
cv2.imwrite("coins_otsu.png", mask)
```

Expected Output: Otsu chose: ~108 and coins_otsu.png showing clean coin separation from background.

Example 2: Bimodal vs unimodal histograms

```
import cv2, numpy as np
import matplotlib.pyplot as plt
from skimage.data import coins

bimodal = coins()                                          # clear background + coins
unimodal = (np.random.rand(300, 400) * 255).astype(np.uint8)

for name, img in [("bimodal", bimodal), ("unimodal", unimodal)]:
    plt.figure()
    plt.hist(img.ravel(), 256, [0, 256])
    plt.title(name)
    plt.show()
    otsu, _ = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    print(f"{name:10s} Otsu={otsu}")
```

Expected Output: Bimodal histogram shows two clear humps; unimodal is flat. Otsu still returns a value for the unimodal case (~127) but the result is meaningless.

Example 3: Pre-blur for stability

```
import cv2
from skimage.data import coins

img = coins()
blur = cv2.GaussianBlur(img, (5, 5), 0)
otsu1, mask1 = cv2.threshold(img,  0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
otsu2, mask2 = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print(f"no blur: {otsu1}    with blur: {otsu2}")
```

Expected Output: Both threshold values are close (~108); blur typically nudges things a few units.

Example 4: Multi-level Otsu via skimage

```
import numpy as np
from skimage.data import camera
from skimage.filters import threshold_multiotsu

img = camera()
thresholds = threshold_multiotsu(img, classes=3)
print("multi-Otsu thresholds:", thresholds)
labels = np.digitize(img, bins=thresholds)
print("classes per pixel range:", labels.min(), labels.max())
```

Expected Output: Two threshold values printed (e.g. [58, 142]); labels is 0/1/2 indicating which intensity class each pixel belongs to.

4. Parameter Sensitivity

Param	Effect
pre-blur σ	bigger = more stable threshold, less detail
number of classes (multi-Otsu)	2 (single Otsu) for binary; 3+ for richer segmentation

5. Common Mistakes

Mistake	What Happens	Fix
Pass non-zero threshold	OpenCV ignores it but it's confusing	Use 0 as the argument
Use Otsu on uneven lighting	Big chunks fail	Use adaptive threshold
Use Otsu on unimodal	Meaningless threshold	Check histogram first

6. Practice Tasks

- Otsu on coins(); print chosen value.
- Plot histograms of coins() vs uniform-random; observe bimodality difference.
- Pre-blur with σ=2; compare Otsu threshold values.

7. Key Takeaways

- Otsu picks the threshold by maximising between-class variance.
- Pass 0 to cv2.threshold with THRESH_OTSU flag.
- Works on bimodal histograms; fails on unimodal or uneven lighting.
- Pre-blur slightly for stability.

8. What's Next

Adaptive thresholding — for images with uneven lighting where one global value can't work.

Lesson 3: Adaptive Thresholding

Module: 11 — Segmentation & Thresholding

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Apply cv2.adaptiveThreshold with ADAPTIVE_THRESH_MEAN_C and _GAUSSIAN_C.
- Tune blockSize and C.
- Recognise when adaptive beats Otsu (uneven lighting).

Prerequisites

- Module 11 Lesson 2

1. Why This Matters

Global thresholds — even Otsu — fail when one side of an image is brighter than the other. Adaptive thresholding picks a local threshold per region, perfect for documents photographed under varying light.

2. Concept

2.1 How it works

For each pixel, look at its neighbourhood (e.g. 11×11), compute a local threshold value, and compare. Neighbours brighter than the pixel → pixel becomes foreground; darker → background.

The local threshold is:

- MEAN_C: mean(neighbours) − C
- GAUSSIAN_C: weighted Gaussian mean − C

C is a small constant subtracted to tune sensitivity.

2.2 OpenCV API

```
out = cv2.adaptiveThreshold(
    src, maxval, adaptiveMethod, thresholdType, blockSize, C)
```

- adaptiveMethod: cv2.ADAPTIVE_THRESH_MEAN_C or _GAUSSIAN_C.
- thresholdType: THRESH_BINARY or THRESH_BINARY_INV.
- blockSize: odd, ≥ 3. Size of the neighbourhood.
- C: integer subtracted from mean. Larger C = stricter (less foreground).

2.3 Picking blockSize

- Too small (3-7): noisy mask, every speck triggers.

- Just right (11-25): captures local features without noise.
- Too large (50+): approaches global thresholding.

The rule: blockSize should be larger than the features you want to detect.

2.4 Picking C

Typical values 2-10. Start at 5 and tune.

2.5 MEAN vs GAUSSIAN

GAUSSIAN tends to look smoother (less staircasing on edges). Use it as the default.

2.6 The classic use case

Old paper photographed with uneven flash:

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
out = cv2.adaptiveThreshold(gray, 255,
                            cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                            cv2.THRESH_BINARY, 25, 8)
```

Text comes out black on white, regardless of which corner of the paper is in shadow.

3. Code Examples

Example 1: Compare on uneven-lit text

```
import cv2, numpy as np
from skimage.data import text

img = text()
# Simulate uneven lighting
h, w = img.shape
ramp = np.tile(np.linspace(0.4, 1.4, w), (h, 1))
uneven = np.clip(img * ramp, 0, 255).astype(np.uint8)
cv2.imwrite("uneven.png", uneven)

_, otsu = cv2.threshold(uneven, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
adapt = cv2.adaptiveThreshold(uneven, 255,
                              cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                              cv2.THRESH_BINARY, 25, 10)

cv2.imwrite("uneven_otsu.png",  otsu)
cv2.imwrite("uneven_adapt.png", adapt)
```

Expected Output: otsu fails — bright side mostly white, dark side mostly black. adapt produces clean text across the whole image.

Example 2: blockSize sensitivity

```
import cv2
from skimage.data import text

img = text()
```

```
for bs in [3, 11, 25, 51]:
    out = cv2.adaptiveThreshold(img, 255,
                                cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                cv2.THRESH_BINARY, bs, 8)
    cv2.imwrite(f"adapt_bs{bs}.png", out)
```

Expected Output: bs=3 is noisy; bs=25 captures text cleanly; bs=51 starts to behave like global.

Example 3: C sensitivity

```
import cv2
from skimage.data import text

img = text()
for c in [0, 5, 10, 20]:
    out = cv2.adaptiveThreshold(img, 255,
                                cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                cv2.THRESH_BINARY, 25, c)
    cv2.imwrite(f"adapt_c{c}.png", out)
```

Expected Output: Low C: more foreground (text + noise). High C: less foreground (text might thin out).

Example 4: MEAN vs GAUSSIAN

```
import cv2
from skimage.data import text

img = text()
m = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
                          cv2.THRESH_BINARY, 25, 10)
g = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                          cv2.THRESH_BINARY, 25, 10)
cv2.imwrite("adapt_mean.png", m)
cv2.imwrite("adapt_gauss.png", g)
```

Expected Output: Both capture text; GAUSSIAN looks slightly smoother / less jaggy.

4. Parameter Sensitivity

Param	Effect
blockSize	small = noisy / fine; large = global-like
C	small = more foreground; large = stricter
MEAN vs GAUSSIAN	GAUSSIAN smoother by a small margin

5. Common Mistakes

Mistake	What Happens	Fix
Even blockSize	Error	Use odd
blockSize too small	Noise everywhere	11+ for typical images
C = 0	Foreground includes noise	Try 5-10
Use on uniform-lit image	OK but no advantage over Otsu	Use simpler method

6. Practice Tasks

- Apply adaptive Gaussian on text() with blockSize=25, C=10.
- Compare to Otsu on the same image with simulated uneven lighting.
- Tune blockSize ∈ {3, 11, 25, 51} and pick a good value.

7. Key Takeaways

- cv2.adaptiveThreshold picks per-region threshold = mean − C.
- Use GAUSSIAN_C as default; blockSize odd, larger than features.
- Best for uneven lighting (documents, scans).
- C tunes how strict the foreground assignment is.

8. What's Next

Watershed — handle touching objects that thresholding alone can't separate.

Lesson 4: Watershed

Module: 11 — Segmentation & Thresholding

Duration: 35 minutes

Difficulty: Intermediate-Advanced

Learning Objectives

- Run the OpenCV watershed pipeline.
- Build sure foreground / sure background / unknown markers.
- Use distance transform to seed markers.

Prerequisites

- Module 11 Lesson 3, morphology (M10).

1. Why This Matters

Touching objects (overlapping coins, cells, packed nuts) merge into one blob after thresholding. Watershed treats the image as a topographic surface and "floods" basins from marker seeds — separating touching objects into distinct labels.

2. Concept

2.1 The intuition

Imagine the inverse of the image as a 3D landscape: bright spots are valleys, dark spots are hills. Drop water into each marker; water flows downhill and meets at "watershed lines" — the boundaries between objects.

2.2 The OpenCV pipeline

- Threshold (often Otsu) → rough binary mask.
- Clean with morphology (open / close).
- Sure background = dilation of mask.
- Distance transform of mask; threshold high values → sure foreground (object cores).
- Unknown = sure_bg − sure_fg.
- Connected components on sure_fg → marker labels.
- Add 1 to all markers (so background isn't 0); set unknown region to 0.
- cv2.watershed(image_color, markers) — modifies markers in place: -1 along watershed lines, integer label elsewhere.

2.3 Distance transform

cv2.distanceTransform(mask, distanceType, maskSize) — value at each white pixel = distance to nearest black pixel. Object cores (far from any boundary) get the largest values; edge pixels get small values.

```
dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
```

Threshold the top X% of dist to get sure cores.

2.4 Why "markers"?

Watershed needs to know where every object's "seed" is. Markers = integer-labeled image: 0 for unknown, 1 for background, 2, 3, 4, ... for each object.

2.5 OpenCV API

```
cv2.watershed(img_bgr, markers)
```

- img_bgr must be 3-channel.
- markers is int32, modified in place.
- After: pixels at watershed boundaries have value -1; other pixels have their marker label.

3. Code Examples

Example 1: Full watershed pipeline on touching circles

```
import cv2, numpy as np

# Build 3 touching circles
img = np.full((300, 600, 3), 240, dtype=np.uint8)
cv2.circle(img, (200, 150), 80, (50, 50, 50), -1)
cv2.circle(img, (300, 150), 80, (50, 50, 50), -1)
cv2.circle(img, (400, 150), 80, (50, 50, 50), -1)
cv2.imwrite("ws_input.png", img)

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

# Cleanup
se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, se, iterations=2)

# sure bg
sure_bg = cv2.dilate(mask, se, iterations=3)

# sure fg via distance transform
dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist, 0.5 * dist.max(), 255, 0)
sure_fg = sure_fg.astype(np.uint8)

unknown = cv2.subtract(sure_bg, sure_fg)

n, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0

cv2.watershed(img, markers)

out = img.copy()
out[markers == -1] = (0, 0, 255)         # watershed lines in red
```

```
cv2.imwrite("ws_output.png", out)
print("number of objects found:", n - 1)
```

Expected Output: ws_output.png shows the 3 circles with red boundary lines between them — successful separation. Console prints number of objects found: 3.

Example 2: Watershed on touching coins

```
import cv2, numpy as np
from skimage.data import coins

gray = coins()
img = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)

_, mask = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, se, iterations=2)

sure_bg = cv2.dilate(mask, se, iterations=3)
dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist, 0.5 * dist.max(), 255, 0)
sure_fg = sure_fg.astype(np.uint8)
unknown = cv2.subtract(sure_bg, sure_fg)

n, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0

cv2.watershed(img, markers)
out = img.copy()
out[markers == -1] = (0, 0, 255)
cv2.imwrite("coins_ws.png", out)
print("coins detected:", n - 1)
```

Expected Output: Coins outlined in red where they touch — typically separates most of them correctly.

Example 3: Distance transform visualisation

```
import cv2, numpy as np
from skimage.data import coins

_, mask = cv2.threshold(coins(), 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
dist_vis = cv2.normalize(dist, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
cv2.imwrite("coins_dist.png", dist_vis)
print("max distance:", dist.max())
```

Expected Output: coins_dist.png is a grayscale image where coin centres are bright (far from edge) and coin boundaries are dark. Max distance ≈ coin radius in pixels.

4. Parameter Sensitivity

Param	Effect

Distance threshold (% of max)	Too low: cores merge again; too high: small objects missed
Cleanup iterations	Too few: noise leaks in; too many: cores too small

5. Common Mistakes

Mistake	What Happens	Fix
Pass grayscale to cv2.watershed	Error	Convert to BGR first
Marker dtype not int32	Error	markers.astype(np.int32)
Marker label 0 used for an object	Conflict with "unknown"	Add 1 to all markers
Skip distance transform; just use mask	Touching objects stay merged	Use distance + threshold for seeds

6. Practice Tasks

- Build 3 touching circles; run watershed pipeline; verify 3 objects.
- Apply to binarised coins(); count separated objects.
- Visualise the distance transform of any mask.

7. Key Takeaways

- Watershed needs markers: background, sure foreground, unknown.
- Distance transform gives object "cores" for sure-foreground seeds.
- cv2.watershed(bgr, markers_int32) modifies markers in place.
- Watershed lines = pixels where markers == -1.

8. What's Next

GrabCut — interactive foreground extraction.

Lesson 5: GrabCut (Interactive Foreground)

Module: 11 — Segmentation & Thresholding

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Use cv2.grabCut for one-click foreground extraction.
- Provide a bounding-box init.
- Refine with mask input.

Prerequisites

- Module 11 Lesson 4

1. Why This Matters

GrabCut produces clean foreground masks from a rough bounding box (or scribbles). It's the engine behind tools like "magic wand" in image editors and is far more sophisticated than simple thresholding.

2. Concept

2.1 The algorithm (very brief)

GrabCut models foreground / background as Gaussian Mixture Models on color, then uses graph cuts to find the optimal segmentation under those models. Iterates 5-10 times for convergence.

2.2 OpenCV API — rectangle init

```
mask = np.zeros(img.shape[:2], dtype=np.uint8)
bgd_model = np.zeros((1, 65), dtype=np.float64)
fgd_model = np.zeros((1, 65), dtype=np.float64)
rect = (x, y, w, h)               # bounding box around target
cv2.grabCut(img, mask, rect, bgd_model, fgd_model, 5, cv2.GC_INIT_WITH_RECT)
```

After running:

```
mask_final = np.where((mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD), 255,
0).astype(np.uint8)
```

2.3 Mask values

Constant	Meaning
cv2.GC_BGD (0)	definite background
cv2.GC_FGD (1)	definite foreground
cv2.GC_PR_BGD (2)	probable background
cv2.GC_PR_FGD (3)	probable foreground

2.4 Refinement

After the initial run, you can paint into the mask manually (definite FG / BG) and re-run with cv2.GC_INIT_WITH_MASK:

```
mask[manual_fg] = cv2.GC_FGD
mask[manual_bg] = cv2.GC_BGD
cv2.grabCut(img, mask, None, bgd_model, fgd_model, 5, cv2.GC_INIT_WITH_MASK)
```

2.5 Use cases

- Cutout for product photos.
- Replacing backgrounds (poor man's green-screen).
- Initial mask for further refinement / training data.

2.6 Limitations

- Slow (several seconds for HD).
- Needs good initial input — randomly placed rect won't help.
- Struggles with similar foreground / background colors.

3. Code Examples

Example 1: GrabCut on astronaut with bounding box

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
mask = np.zeros(img.shape[:2], dtype=np.uint8)
bgd, fgd = np.zeros((1, 65), np.float64), np.zeros((1, 65), np.float64)
rect = (60, 30, 380, 460)        # x, y, w, h covering the astronaut
cv2.grabCut(img, mask, rect, bgd, fgd, 5, cv2.GC_INIT_WITH_RECT)

m_final = np.where((mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD), 255,
0).astype(np.uint8)
out = cv2.bitwise_and(img, img, mask=m_final)
cv2.imwrite("astro_grabcut.png", out)
cv2.imwrite("astro_mask.png", m_final)
```

Expected Output: astro_grabcut.png shows the astronaut cleanly extracted with the background blackened.

Example 2: Replace background after GrabCut

```
import cv2, numpy as np
from skimage.data import astronaut, coffee

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
mask = np.zeros(img.shape[:2], dtype=np.uint8)
bgd, fgd = np.zeros((1, 65), np.float64), np.zeros((1, 65), np.float64)
cv2.grabCut(img, mask, (60, 30, 380, 460), bgd, fgd, 5, cv2.GC_INIT_WITH_RECT)
m = np.where((mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD), 255,
0).astype(np.uint8)
```

```
new_bg = cv2.resize(cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR), (img.shape[1],
img.shape[0]))
inv = cv2.bitwise_not(m)
fg = cv2.bitwise_and(img, img, mask=m)
bg = cv2.bitwise_and(new_bg, new_bg, mask=inv)
comp = cv2.add(fg, bg)
cv2.imwrite("astro_on_coffee.png", comp)
```

Expected Output: Astronaut placed over the coffee photo.

Example 3: Iterations matter

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
rect = (60, 30, 380, 460)
for it in [1, 3, 5, 10]:
    mask = np.zeros(img.shape[:2], dtype=np.uint8)
    bgd, fgd = np.zeros((1, 65), np.float64), np.zeros((1, 65), np.float64)
    cv2.grabCut(img, mask, rect, bgd, fgd, it, cv2.GC_INIT_WITH_RECT)
    m = np.where((mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD), 255,
0).astype(np.uint8)
    cv2.imwrite(f"grabcut_it{it}.png", m)
```

Expected Output: 1 iteration: rough mask. 10 iterations: cleaner mask. Returns plateau around 5 iterations.

4. Parameter Sensitivity

Param	Effect
Rectangle accuracy	Tight rect = better result
Iterations (5 default)	More = better, slow
Mask-mode refinement	Adds user knowledge → big quality bumps

5. Common Mistakes

Mistake	What Happens	Fix
Rect outside or too tight	Bad mask	Encompass target with a small margin
Apply on grayscale	Error	Must be 3-channel BGR
Reusing bgd_model / fgd_model from prior call	Tainted state	Reinitialise to zeros for each new image
Expecting perfect cut on similar fg/bg colors	Mask leaks	Switch to mask init + manual scribbles

6. Practice Tasks

- Run GrabCut on astronaut() with the rect from Example 1. Save mask + extracted FG.
- Composite the astronaut onto coffee() as a new background.
- Try iterations 1, 3, 5, 10; compare visually.

7. Key Takeaways

- GrabCut = graph-cut + GMM-based interactive foreground.
- Initialise with a bounding box, then run 5+ iterations.
- Final mask = (FGD | PR_FGD).
- Slow but high-quality; pair with mask-mode refinement for tough cases.

8. What's Next

End of Module 11. Next: Module 12 — Contours & Shape Analysis — analyse the regions you just segmented.

Module 11 — Exercises

15 exercises.

11.1 Thresholding

- (E) Threshold coins() at 80; save mask.
- (M) Apply all 5 threshold types on camera().
- (H) Use mask to keep only the coins; blacken background.

11.2 Otsu

- (E) Otsu on coins(); print chosen value.
- (M) Compare bimodal vs unimodal histograms.
- (H) Use skimage threshold_multiotsu with 3 classes.

11.3 Adaptive

- (E) Adaptive Gaussian on text() (blockSize=25, C=10).
- (M) Simulate uneven lighting; compare Otsu vs adaptive.
- (H) Tune blockSize $\in \{3,11,25,51\}$; pick a good one.

11.4 Watershed

- (E) Build 3 touching circles; run full watershed; expect 3 objects.
- (M) Apply watershed to binarised coins().
- (H) Visualise distance transform of any mask.

11.5 GrabCut

- (E) GrabCut on astronaut() with bounding box; save mask + extracted FG.
- (M) Composite onto coffee() background.
- (H) Try iterations 1, 3, 5, 10; compare.

Module 11 — Quiz

10 MCQs.

Q1

cv2.threshold(src, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU) returns: A. only the mask B. (otsu_value, mask) C. only Otsu value D. error

Answer: B (L2)

Q2

Otsu works well when the histogram is: A. flat B. unimodal C. bimodal D. random

Answer: C (L2)

Q3

For uneven-lit text scans, the best threshold is: A. fixed 128 B. Otsu C. adaptive D. random

Answer: C (L3)

Q4

adaptiveThreshold blockSize must be: A. even B. odd, ≥ 3 C. ≥ 51 D. any

Answer: B (L3)

Q5

In adaptive threshold, larger C means: A. more foreground B. less foreground C. blurrier D. inverted

Answer: B (L3)

Q6

Watershed separates touching objects by treating the image as: A. random B. text C. topographic surface D. graph

Answer: C (L4)

Q7

For watershed seeds, the distance transform value at a pixel is: A. distance to centre B. distance to nearest black pixel C. brightness D. gradient

Answer: B (L4)

Q8

GrabCut needs the image as: A. grayscale B. binary C. BGR (3 channel) D. RGBA

Answer: C (L5)

Q9

After GrabCut, the foreground mask is: A. mask == 1 B. (mask == FGD) | (mask == PR_FGD) C. mask == 0 D. mask > 128

Answer: B (L5)

Q10

For "old paper photographed with uneven flash", the right segmentation method is: A. simple threshold B. Otsu C. adaptive D. GrabCut

Answer: C (L3)

Module 11 — Assignment: Coin Counter

Estimated time: 2 hours

Combines: Module 11 + Modules 9, 10

Brief

Build a CLI tool that counts the coins in an image using full segmentation pipeline (threshold → morphology → watershed).

Requirements

- count.py INPUT [--output OUT]
- Read as grayscale.
- Otsu threshold.
- Open + close cleanup.
- Distance transform + watershed.
- Count distinct labels (excluding background and watershed lines).
- Draw red watershed boundaries on the original color version.
- Print count, save the annotated result.

Constraints

- Modules 1-11 only.
- Use cv2.watershed.
- Use argparse.

Expected Output

```
$ python count.py datasets/starter/coins.png
detected: 24 objects
saved -> coins_counted.png
```

Grading Rubric

Criterion	Weight
Threshold + cleanup	20%
Distance transform	15%
Watershed correct	30%
Count + annotation	20%
Style	15%

Submission

Save as assignment_module11.py.

Module 11 — Mini Project: One-Click GrabCut Cutout

Overview

User clicks two opposite corners of a bounding box around an object; GrabCut runs and produces a cleanly cut-out PNG with transparent background.

Concepts Used

- GrabCut (M11 L5)
- Mouse callbacks (M2 L3)
- Alpha PNG output (M2 L6)

Features

- cutout.py INPUT [--output OUT.png]
- User clicks 2 corners (top-left, bottom-right).
- GrabCut runs 5 iterations on that rect.
- Result saved as PNG with alpha (transparent background).

Starter Code

cutout.py:

```
import sys, cv2, numpy as np

pts = []

def on_mouse(event, x, y, flags, param):
    global pts
    if event == cv2.EVENT_LBUTTONDOWN:
        pts.append((x, y))
        cv2.circle(param, (x, y), 5, (0, 255, 0), -1)

def run(path, out):
    img = cv2.imread(path)
    if img is None: raise FileNotFoundError(path)
    if max(img.shape) > 900:
        img = cv2.resize(img, None, fx=0.6, fy=0.6)
    disp = img.copy()
    cv2.namedWindow("click 2 corners")
    cv2.setMouseCallback("click 2 corners", on_mouse, disp)
    while True:
        cv2.imshow("click 2 corners", disp)
        if len(pts) >= 2: break
        if cv2.waitKey(20) & 0xFF == ord('q'): return
    cv2.destroyAllWindows()

    x1, y1 = pts[0]; x2, y2 = pts[1]
    rect = (min(x1, x2), min(y1, y2), abs(x2 - x1), abs(y2 - y1))

    mask = np.zeros(img.shape[:2], dtype=np.uint8)
```

```
    bgd, fgd = np.zeros((1, 65), np.float64), np.zeros((1, 65), np.float64)
    cv2.grabCut(img, mask, rect, bgd, fgd, 5, cv2.GC_INIT_WITH_RECT)
    m = np.where((mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD), 255,
0).astype(np.uint8)

    # Build BGRA result with alpha mask
    bgra = cv2.cvtColor(img, cv2.COLOR_BGR2BGRA)
    bgra[..., 3] = m
    cv2.imwrite(out, bgra)
    print("saved", out)

if __name__ == "__main__":
    path = sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/astronaut.png"
    out  = sys.argv[2] if len(sys.argv) > 2 else "cutout.png"
    run(path, out)
```

Expected Interaction

```
$ python cutout.py datasets/starter/astronaut.png
(window opens; click top-left of astronaut, then bottom-right)
saved cutout.png
```

cutout.png is a transparent PNG with the astronaut on transparent background.

Extension Challenges

- Allow rectangle drag (instead of two clicks).
- After initial mask, allow scribble refinement (LMB=FG, RMB=BG) and re-run.
- Output a separate alpha-only PNG for further use.

Grading Rubric

Criterion	Weight
Two-click rect input	25%
GrabCut correct	30%
BGRA save with alpha	20%
Code organisation	15%
Style	10%